Draw It or Lose It

CS 230 Project Software Design Template
Version 1.2

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	02/07/2026	Jaiden Logan	Initial draft of software design template for Draw It or Lose It web-based version.
1.1	02/10/2026	Jaiden Logan	Minor edits; Previous iteration was already updated to the standards for Project Two
1.2	02/22/2026	Jaiden Logan	Polished recommendations panel for Project Three submission

Executive Summary

The Gaming Room wants to expand Draw It or Lose It from an Android-only app into a web-based game that can run across multiple platforms. The main needs are keeping game, team, and player names unique, supporting multiple teams and players, and making sure the system can manage multiple game sessions without creating duplicates when users save or create new games.

To solve this cleanly, the design uses a singleton GameService class as the single "manager" that controls game creation and lookup. The service also loops through the list to check for existing names, existing games (and later teams/players) before adding anything new, which is what prevents duplicates. The domain model is built around a shared Entity base class so all core objects (Game, Team, Player) have a consistent id and name, which makes the code simpler to maintain and easier to scale as the project grows.

Requirements

The game application must support multiple games running at once, where each game can include one or more teams and each team can include multiple players. Game names and team names must be unique so the application can check whether a name is already taken when a user is creating something new. The system must also ensure only one GameService exists in memory at a time, since it acts as the central place that tracks and manages game instances. Finally, the design needs to be able to work in a distributed web environment where clients connect over a network, meaning reliability, security, and performance have to be considered early.

Design Constraints

A web-based distributed environment creates a few built-in constraints. First, the application has to handle multiple users connecting at the same time, which means shared data (like lists of games, teams, and players) must be managed carefully to avoid duplicates and conflicts. Second, network communication isn't guaranteed to be fast or stable, so the design should assume latency, disconnects, and retries can happen.

Another constraint is platform variety: users could be on Windows, Mac, Linux, or mobile devices, so the client side should avoid platform-specific dependencies. That's a big reason a browser-based front end makes sense, since it keeps client requirements light and consistent.

Finally, security matters more in a web app than a local app because user traffic and game data move over networks. The design needs secure authentication options, encrypted connections (TLS/HTTPS), and safe handling of user input to reduce common web risks.

System Architecture View

This project is designed around a simple client/server approach: the client runs in a web browser and communicates with a server-hosted game service over the network. The server side would manage game sessions, enforce unique names, and store game/team/player data using a database or persistent storage as the project expands.

Domain Model

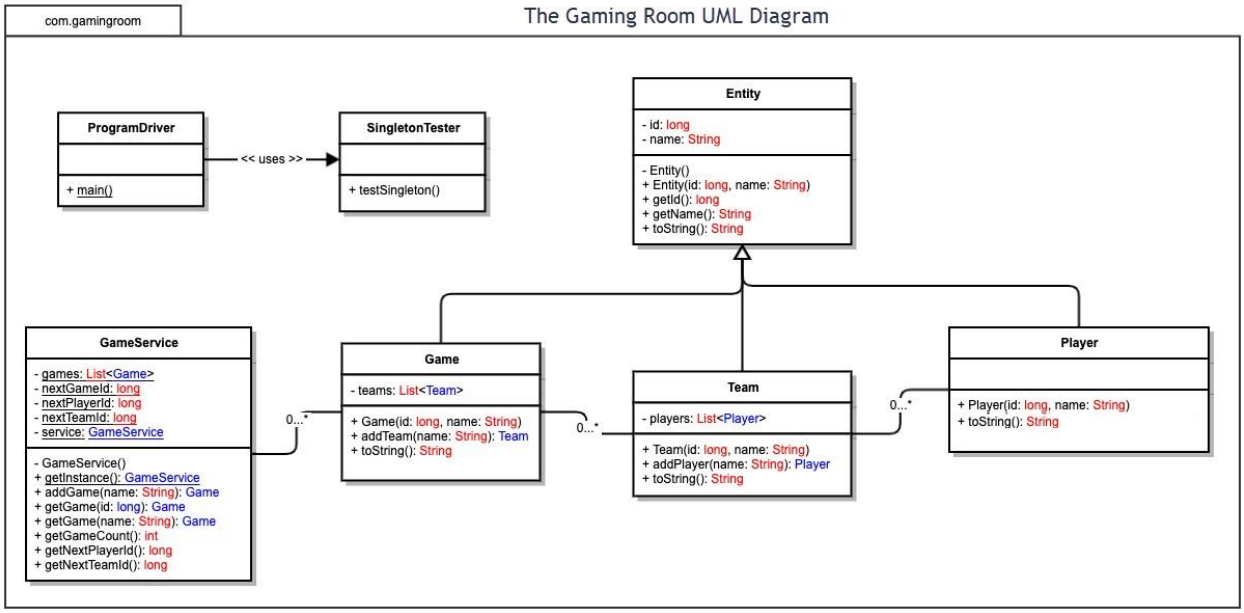
The UML domain model is structured around a base Entity class that contains shared fields like id and name. That base class is reused by the Game, Team, and Player classes, which cuts down repeated code

and keeps naming/ID behavior consistent across the whole system. This is a straightforward example of inheritance and code reuse, since the shared “identity” logic lives in one place.

A Game contains a list of Team objects, and each Team contains a list of Player objects. That represents the real-world relationships clearly: teams belong to a game, and players belong to a team. This also supports the requirements that a game can have multiple teams and each team can have multiple players.

The GameService class acts like the main manager for creating and retrieving games. It’s built using the singleton pattern, meaning only one instance of GameService is allowed in memory. That matters because it ensures there’s one trusted place that tracks all active games and controls ID assignment. The iterator pattern is used when adding or searching for items (like games) so the code can loop through existing objects to confirm uniqueness before creating a new one.

Overall, the diagram shows key OOP ideas like encapsulation (data is accessed through methods), inheritance (Entity → Game/Team/Player), and composition (Game has Teams, Team has Players), and those choices line up directly with the project’s requirements.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client’s requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS can host a web server for development, but it's not the most common choice for production hosting. It works fine for testing and local builds, but most real deployments still lean toward Linux-based servers for cost and tooling.	Linux is the most practical server option because it's common in cloud hosting, stable under load, and has strong support for web stacks and automation. It's also cost-friendly and easy to scale.	Windows Server can host web applications well, especially in Microsoft-based environments, but it can cost more and may require more system resources depending on the stack. It's still a solid option if the team is already Windows-focused.	Mobile devices aren't realistic as servers for a hosted web game. They're limited in uptime, network reliability, and performance, and aren't meant for always-on services.
Client Side	For a browser-based client, Mac is a strong option because it runs modern browsers well and doesn't need special setup beyond internet access. The main consideration is just testing across Safari/Chrome/Firefox to avoid browser-specific issues.	Linux works great for a web client, especially because browsers are well supported. The only real concern is making sure the UI behaves consistently across different desktop environments and browser versions.	Windows is the most common desktop environment for users, so it's important to test here. A browser-based client keeps things simple because users don't need to install anything.	Mobile support is important since many users will play on phones/tablets. This adds constraints like touch controls, different screen sizes, and performance limits, so responsive UI design matters.
Development Tools	macOS supports Java tooling well, and IDEs like Eclipse or IntelliJ run smoothly. It's also a common environment for web development tools and testing across browsers.	Linux is excellent for development because it's friendly for command-line tools, builds, and automation. Java, Git, and server tooling fit naturally here, which can make deployment easier later.	Windows is a very common development environment for students and teams. Eclipse, Java, and Git work fine, and it's easy to test the client side since many users will also run Windows.	For mobile development you'd usually use Android Studio or Xcode, but since this project is web-based, mobile devices mainly matter for testing rather than building the actual application directly on them.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: I would recommend hosting the server side on Linux (like Ubuntu Server) because it's widely supported in cloud environments, cost-effective, and reliable for web hosting. Since the client is web-based, users can still play from Windows, Mac, Linux, or mobile without needing a platform-specific build.

2. **Operating Systems Architectures**: A Linux server can run on common 64-bit architectures (x86_64), and cloud providers also offer ARM64 options. Linux follows a kernel + user space design, where the kernel handles core tasks like process scheduling, memory management, and networking, and the game server runs in user space. This is useful for a server-hosted web app because Linux is built to handle many concurrent processes/threads efficiently, and it also works well with virtual machines and containers, which makes deployment more consistent. For a web application, the client side doesn't depend heavily on CPU architecture because the browser handles most compatibility issues.

3. **Storage Management**: For persistent storage, a managed relational database like PostgreSQL is a good fit because it handles structured data (games, teams, players) cleanly and supports safe constraints like uniqueness. For early stages, in-memory lists work for a prototype, but a database becomes necessary once multiple users and saving/loading are involved. A database also makes it easier to support backups and recovery so game data is not lost.

4. **Memory Management**: The server should control memory by keeping only the needed game session data active and persisting the rest to storage. Using the singleton GameService helps here because it avoids duplicate service instances and keeps state management centralized. To keep memory from growing over time, the server should also clean up inactive sessions (for example, using timeouts) and avoid holding unnecessary references that could cause memory leaks in a long-running server.

5. **Distributed Systems and Networks**: A standard approach would be a web client that communicates to the server using HTTP/HTTPS. Since connectivity issues can happen, the design should account for timeouts, retries, and reconnects so the app stays usable when the network is slow or drops. For scale and reliability, the system could later add load balancing and caching, since multiple users may connect at once and network conditions may vary. Keeping most shared data in the database also helps when running multiple server instances.

6. **Security**: At minimum, the application should use HTTPS/TLS to encrypt traffic, validate all input to prevent common attacks, and avoid exposing internal IDs or unsafe debug output to users. If accounts are added later, authentication and role-based access become important, along with secure password storage and session handling. On the backend, using least-privilege access for the database and keeping systems patched also helps protect user information.